CHAPTER III

The Meta-Object Operating System Environment

3.1 Architecture

The MOOSE project integrates applied formal methods with object technology to achieve interoperation of heterogeneous distributed systems. The use of formal methods also creates the potential for verification of these complex systems. The architecture, which is depicted in Figure 3.1, is composed of an operational hierarchy and a verification hierarchy. The operational hierarchy is supported by a formal execution model for concurrent processes. The verification hierarchy consists of theorem pools corresponding to each layer in the operational hierarchy.

3.1.1 Operational Hierarchy

Layers in the operational hierarchy represent abstractions achieved through the use of denotational semantics. The foundation of the operational hierarchy serves as a semantic algebra for an intermediate layer of operational abstraction. This intermediate layer expresses primitive distributed object functionality such as encapsulation, message-passing, and even contains a set of capability-based security mechanisms. The distributed object model in the intermediate layer can be used to capture the behavior of virtually any complex object-oriented model.

This intermediate layer provides a semantic algebra for the denotational semantics of more sophisticated object-oriented programming languages and systems. Existing languages (such as Java, Smalltalk and C++) and architectures (such as CORBA and DCE) can be given denotational semantics that map to the intermediate layer. Using a common semantic algebra provides an opportunity for interoperation between heterogeneous components.

Mumbo, a new programming language, is introduced to demonstrate this ap-

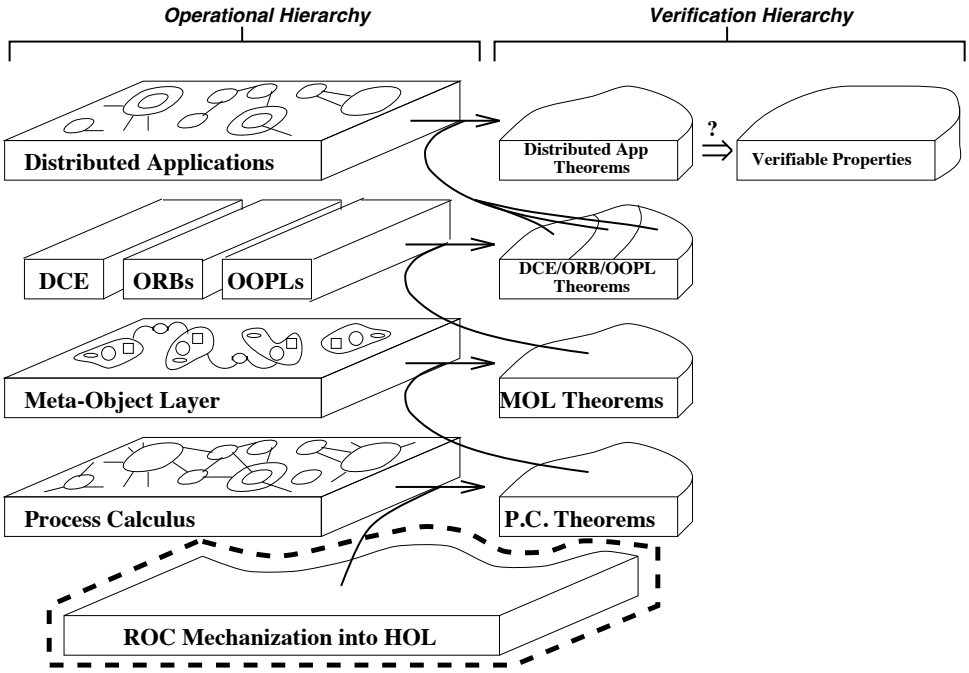

Figure 3.1: Layered semantics of MOOSE.

proach. This object-oriented language is given its semantics in terms of the intermediate layer of the operational hierarchy. It can be used to orchestrate secure interoperation between heterogeneous software components. The security model of the meta-object layer is adopted to support Discretionary Access Control (DAC) and to constrain class and object protocols.

3.1.2 Verification Hierarchy

The operational hierarchy employs a foundational semantics of concurrent processes. The foundation's formal operational semantics are translated into higher order logic [12], a more expressive formal system. This process, known as mechanization, facilitates mechanical reasoning about concurrent processes. The mechanization is a deep-embedding; i.e., only definitions are added to the logic to express behavior of the system. This approach guarantees a consistent logical system.

Once the foundation is mechanized, a pool of theorems must be derived for

the foundational system. A pool of theorems relating to the intermediate layer is derived from the pool of foundational theorems by leveraging the denotational semantics of the intermediate layer. These theorems express the behavior of components at the intermediate level.

This technique is applied repeatedly to construct successive layers in the verification framework. Each superior layer expresses behavior for increasingly abstract layer components. MOOSE approaches application-level reasoning in this manner.

3.2 The Robust Object Calculus

The Robust Object Calculus (ROC) is the foundation of the MOOSE architecture. It is a special process calculus tailored to modeling concurrent objects. ROC is used both as an execution model and as a basis for system verification.

The Robust Object Calculus (ROC) supplies formal operational semantics to all layers in the MOOSE operational hierarchy. ROC supports complex message-passing and a robust form of encapsulation for concurrently executing objects. Furthermore, the concepts of simulation and bisimilarity are definable for ROC systems, demonstrating its viability as a formal system. The operational semantics of ROC are translated into a logical system to support mechanical reasoning. Thus, ROC serves as a foundation for both the operational and the verification hierarchies.

3.2.1 Communication in ROC

Communication in ROC is based on pattern-matching. Complex message-passing is achieved by matching *values* to *patterns*. Two agents may communicate when a value exposed by one matches a pattern exposed by the other. Values are nested tuples of *names*. Patterns are nested tuples of names and *wildcards*. The following notation is used: $a, b, c \in \mathcal{A}$ (agents); $m, n, p \in \mathcal{N}$ (names); $u, v \in \mathcal{V}$ (values); $x, y, z \in \mathcal{X}$ (patterns).

Definition 3.2.1 *A* value *is a name, an agent, a tuple of values, or a bindable value. A value does not contain wildcards at any level. The BNF definition of a value is:*

$$v ::= n \mid a \mid [v_1, v_2, ..., v_j] \mid v\#.$$

Definition 3.2.2 *A* pattern *is a value, wildcard, or tuple of patterns. The BNF definition of a pattern is:*

$$x ::= v \mid n? \mid [x_1, x_2, ..., x_i].$$

A wildcard, i.e., a placeholder for a value, is denoted by $n?$ ($n \in \mathcal{N}$). A *bindable value*, which can be bound to a placeholder, is denoted by $n\#$ ($n \in \mathcal{N}$). Bindable values are those values which are transmittable to agents with matching wildcard patterns. Values not tagged with "#" are called *unbindable values*. They cannot be matched by a wildcard.

3.2.2 Pattern-Matching

The matching of patterns and values is accomplished by the "$\sim$" operator. The semantics of the bindable symbol "#" are clarified in the matching rules below. Unbindable values can match bindable values, but cannot match wildcards.

Definition 3.2.3 *The* match *operator is denoted by "$\sim$". The matching rules are:*

$$\begin{array}{lcl@{\qquad\qquad}lcl} v\# & \sim & v\# & v & \sim & v \\ v\# & \sim & v & v & \sim & v\# \\ v\# & \sim & n? & \vec{v}(\#) & \sim & \vec{x} \Leftrightarrow \forall i, v_i \sim x_i. \end{array}$$

The last matching rule applies to a tuple of values where the tuple is either bindable or unbindable. This condition is denoted by "(#)".

Pattern-matching is achieved by applying the rules given above. For example, $[m, [n]] \sim [m, [n]]$. On the other hand, $[m, n] \not\sim [m, [n]]$.

Wildcards can match bindable values. For example, $[m,[n]\#] \sim [m,[n]]$ and $[m,[n]\#] \sim [m,p?]$. On the other hand, $[m,[n]] \not\sim [m,p?]$.

Free occurrences of the wildcard are replaced with the value inside the accepting agent. When $v \sim x$, the notation $a\{v/x\}$ is used to denote that all wildcards in x are replaced by the corresponding subvalues of v in the agent a. For example, when $v\#$ binds to $n?$ in $n? \rightarrow a$, each free occurrence of n in a is replaced by v.

3.2.3 ROC Syntax

The ROC syntax is derived from Nierstrasz's Object Calculus (OC) [50]. The symbol "#" distinguishes bindable values from unbindable values. The non-deterministic choice operator "+" from π-calculus is added along with "$*$", a new left-preferential composition operator. The BNF syntax is shown below.

$$
\begin{array}{rll}
a ::= & a \ \& \ a & \text{(concurrent composition)} \\
| & a * a & \text{(left preferential composition)} \\
| & n := a & \text{(recursion)} \\
| & a + a & \text{(non-deterministic choice)} \\
| & a \mid a & \text{(left preferential choice)} \\
| & x \rightarrow a & \text{(input)} \\
| & v\hat{}a & \text{(output)} \\
| & a@v & \text{(application)} \\
| & n\backslash a & \text{(new name } n \text{ in } a\text{)} \\
| & n & \text{(name)} \\
| & \mathbf{nil} & \text{(empty agent)}
\end{array}
$$

The following notation is used in the syntax definition: $a, b, c \in \mathcal{A}$ (agents); $n, m \in \mathcal{N}$ (names); $v \in \mathcal{V}$ (values); $x, y, z \in \mathcal{X}$ (patterns). Note that the set of values is a proper subset of the set of patterns, i.e., $\mathcal{V} \subset \mathcal{X}$. The operators "$*$", "&", ":=", "|", "+", "$\rightarrow$", "^" and "\" are right-associative. This order indicates the binding precedence from loosest to tightest. The application operator, "@", is left-associative. Its binding precedence is tighter than "^" and looser than "\". Agent communication may occur when an input pattern *matches* an output value.

Structural congruence rules are used for manipulating expressions. While they do not represent system activity, they can be used to transform stable expressions

into reducible states. Note that := is overloaded to also mean $\stackrel{def}{\equiv}$.

1. $a \mathbin{\&} b \equiv b \mathbin{\&} a, \quad a \mathbin{\&} (b \mathbin{\&} c) \equiv (a \mathbin{\&} b) \mathbin{\&} c$
2. $a + b \equiv b + a, \quad a + (b + c) \equiv (a + b) + c$
3. $n := a \equiv a\{(n := a)/n?\}$
4. $n\backslash a \equiv a, n \notin fn(a)$
5. $n\backslash m\backslash a \equiv m\backslash n\backslash a$
6. $n\backslash a \star b \equiv n\backslash(a \star b), n \notin fn(b), \quad a \mid n\backslash b \equiv n\backslash(a \mid b), n \notin fn(a)$
   where $\star \in \{\&, *, |, +, @\}$
7. $a \mathbin{\&} \mathbf{nil} \equiv a, \quad \mathbf{nil}@v \equiv \mathbf{nil}$
8. $n := a \equiv n' := a\{n'/n?\}, n' \notin fn(a)$
9. $n\backslash a \equiv n'\backslash a\{n'/n?\}, n' \notin fn(a)$
10. $x \rightarrow a \equiv x\{n'/n?\} \rightarrow a\{n'/n?\}, n' \notin fn(x, a)$

Rules 1 and 2 address the commutativity and associativity of parallel composition and choice, respectively. Rule 3 is for expanding recursive agents. Rule 4 stipulates when a restriction can be discarded ($fn(a)$ denotes the set of free names in a). Rule 5 formalizes the commutativity of restriction. Rule 6 describes scope extrusion. Scope extrusion expands the scope of a restriction to proximal agents. Rule 7 shows the effect of **nil** and any value applied to **nil**. Rules 8-10 define α-conversion for agents which substitutes globally unique names for local names. They are useful when scope extrusion is necessary.

3.2.4 Communication and Reduction Rules

Communication and reduction inference rules supply the semantics for agent expressions. Actions in the ROC universe are comprised of communication, reduction and binding. The rules are similar to those in OC [50] except for an additional rule to handle non-deterministic choice.

Definition 3.2.4 Communication offers *are written as* $\stackrel{\alpha}{\longmapsto}$, *where* α *is either* v *(for input) or* $\bar{v}$ *(for output).* Reduction *is written as* $\longmapsto$. *Communication offers and reduction are defined by the following rules:*

$$\textbf{In}: \frac{v \sim x}{x \to a \stackrel{v}{\longmapsto} a\{v/x\}} \qquad \textbf{Out}: \frac{}{v\hat{}a \stackrel{\bar{v}}{\longmapsto} a}$$

$$\textbf{Conc}: \frac{a \stackrel{\alpha}{\longmapsto} a'}{a \,\&\, b \stackrel{\alpha}{\longmapsto} a' \,\&\, b} \qquad \textbf{Choice}: \frac{a \stackrel{\alpha}{\longmapsto} a'}{a + b \stackrel{\alpha}{\longmapsto} a'}$$

$$\textbf{ConcIf}: \frac{a \stackrel{\alpha}{\longmapsto} a'}{a * b \stackrel{\alpha}{\longmapsto} a' * b} \qquad \textbf{Conc_Else}: \frac{OS(a), DNO(a,\alpha), b \stackrel{\alpha}{\longmapsto} b'}{a * b \stackrel{\alpha}{\longmapsto} a * b'}$$

$$\textbf{ChoiceIf}: \frac{a \stackrel{\alpha}{\longmapsto} a'}{a \mid b \stackrel{\alpha}{\longmapsto} a'} \qquad \textbf{Choice_Else}: \frac{OS(a), DNO(a,\alpha), b \stackrel{\alpha}{\longmapsto} b'}{a \mid b \stackrel{\alpha}{\longmapsto} b'}$$

$$\textbf{Comm}: \frac{a \stackrel{v}{\longmapsto} a', b \stackrel{\bar{v}}{\longmapsto} b'}{a \,\&\, b \longmapsto a' \,\&\, b'} \qquad \textbf{Apply}: \frac{a \stackrel{v}{\longmapsto} a'}{a@v \longmapsto a'}$$

$$\textbf{Left}: \frac{a \longmapsto a'}{a \star b \longmapsto a' \star b}, \star \in \{\&, *, |, +, @\} \quad \textbf{Right}: \frac{b \longmapsto b'}{a \star b \longmapsto a \star b'}, \star \in \{\&, *, |, +, \backslash\}$$

$$\textbf{Struct}: \frac{a \equiv b, b \longmapsto b', b' \equiv a'}{a \longmapsto a'}.$$

Conc provides concurrency by allowing "composed" agents to reduce independently. **Choice** allows only one of several possible activities. The **ConcIf** (**ChoiceIf**) and **ConcElse** (**ChoiceElse**) rules prefer the reduction of the left side over the right side under the concurrency (choice) operator. This is important for handling "default" actions (which is lacking in the π-calculus [46]).

The **Else** rules use the decidable predicates $OS(a)$ and $DNO(a,\alpha)$ which require the left agents be *Offer Stable* and at the same time not offer α. $OS(a)$ states that the offers of agent a are stable, so that it is impossible for the agent to further internally evolve and produce new offers, even though the agent may

not be completely stable. It is not necessary to look at any offers beyond guards to satisfy this predicate. It is necessary to look into unguarded applications (@), since an offer may come from within an application if the applied value can be consumed. $DNO(a, \alpha)$ simply means that a cannot offer α.

Comm (global communication) matches complementary communication offers and reduces the system by realizing the communication. **Comm** works in concert with the **In** and **Out** rules. **Apply** (local communication) requires agent a to eventually accept value v. It cannot communicate externally using **Comm** until value v is consumed. If v is never consumed, then agent $a@v$ is effectively dead. The **Left** and **Right** rules allow for activity within (or under) the various operators. The **Struct** rule allows an expression to be manipulated using the structural congruence rules so that further activity can occur when a "stable" agent is structurally congruent to a reducible one. The **Struct** rule manifests symmetrical communication rules for **Conc** and **Choice** where $a \ \& \ b \equiv b \ \& \ a$ and $a + b \equiv b + a$.

3.2.5 ROC and Encapsulation

ROC extends OC by supporting robust agent encapsulation. The encapsulation property is critical to object-oriented systems. It mandates that private services and values in objects be inaccessible to external objects, thereby ensuring that objects have well-defined interfaces. From the point of view of process calculi, this necessitates a higher level of communication control between agents.

The π-calculus [46] achieves this control by using restriction (unique-naming) to create globally unique labels for communication ports. This works because there is no wildcard matching, i.e., only identical ports can match. Unfortunately, this severely hinders its capacity for complex message-passing.

Encapsulation must be guaranteed by the construction of the agent itself and not by assumptions made about external agents. ROC allows such guarantees to be made without sacrificing the ability to model complex message-passing. Unbindable values in ROC make robust agent encapsulation possible.

In general, an agent is encapsulated using restriction to create an unbindable globally unique identifier for the agent. Exposed values containing the unbindable identifier are guaranteed to match only patterns containing the globally unique identifier. Such a pattern can exist outside the encapsulated agent only if the agent has transmitted its identifier to an external agent by making it bindable to an exposed value. Multiple identifiers can be used to achieve nested encapsulation which is essential to most object systems.

3.2.6 Behavioral Equivalence

Theories of behavioral equivalence are important for system specification and verification [45]. By constructing a simple model that is behaviorally equivalent to a more complex model, it is possible to better understand complex system behavior. Furthermore, behavioral equivalence forms the basis of subtyping, interoperability and substitutability [39].

Behavioral equivalence theories are built on formal definitions of simulation and bisimulation. Two basic forms of simulation are defined. One is a strict form of simulation that considers all actions. The other considers only the *observable behavior* of agents. The observable behavior of an agent refers to any communication involving that agent and its environment (which includes all agents external to the agent). Internal communications are not considered observable.

The notion of observational communication is formalized by defining $P \stackrel{\alpha}{\Longrightarrow} Q$, which states that P reduces to Q after some observable communication:

$$P \stackrel{\alpha}{\Longrightarrow} Q \stackrel{\text{def}}{=} P \longmapsto^{*} \stackrel{\alpha}{\longmapsto} \longmapsto^{*} Q.$$

The notation "$\longmapsto^{*}$" indicates an arbitrary number of internal communications (reductions).

Relation predicates of type S are predicates on ROC expression pairs. Strong simulation and weak simulation are predicates that operate on relation predicates of type S. Strong simulation mandates that each potential communication with agent P has a counterpart in Q, i.e., P is simulated by Q, for each P and Q in S.

Weak simulation is based on observable communications. The definitions below formalize notions of strong simulation and weak simulation:

$$S : ROC \rightarrow ROC \rightarrow Bool$$

$$StrongSimulation : (ROC \rightarrow ROC \rightarrow Bool) \rightarrow Bool$$

$$WeakSimulation : (ROC \rightarrow ROC \rightarrow Bool) \rightarrow Bool$$

$$\begin{aligned} StrongSimulation\ S \ \stackrel{\text{def}}{=}\ & \forall P, Q.S\ P\ Q \supset (\forall P'.P \longmapsto P' \supset \exists Q'.Q \longmapsto Q') \wedge \\ & (\forall \alpha.\forall P'.P \stackrel{\alpha}{\longmapsto} P' \supset (\exists Q'.Q \stackrel{\alpha}{\longmapsto} Q' \wedge S\ P'\ Q')) \end{aligned}$$

$$\begin{aligned} WeakSimulation\ S \ \stackrel{\text{def}}{=}\ & \forall P, Q.S\ P\ Q \supset \forall \alpha.\forall P'.P \stackrel{\alpha}{\Longrightarrow} P' \supset \\ & (\exists Q'.Q \stackrel{\alpha}{\Longrightarrow} Q' \wedge S\ P'\ Q'). \end{aligned}$$

The predicates $StronglyBisimilar$ and $WeaklyBisimilar$ convey analogous notions of bisimilarity. The definitions below are patterned after notions of bisimulation found in [45]:

$$StronglyBisimilar : ROC \rightarrow ROC \rightarrow Bool$$

$$WeaklyBisimilar : ROC \rightarrow ROC \rightarrow Bool$$

$$\begin{aligned} StronglyBisimilar\ P\ Q \stackrel{\text{def}}{=} & \exists S.S\ P\ Q \wedge \\ & StrongSimulation\ S\ \wedge StrongSimulation\ (\lambda x\ y.S\ y\ x) \end{aligned}$$

$$\begin{aligned} WeaklyBisimilar\ P\ Q \stackrel{\text{def}}{=} & \exists S.S\ P\ Q \wedge \\ & WeakSimulation\ S\ \wedge WeakSimulation\ (\lambda x\ y.S\ y\ x). \end{aligned}$$

These definitions facilitate mechanical reasoning about the behavioral equivalence of ROC systems. Mechanizing ROC within a more expressive formal system, e.g., HOL, expands the possibilities for system verification. Models and languages given ROC semantics inherit this potential for verification.